UNIVERSITÄT TÜBINGEN
PROF. DR.-ING. HENDRIK P.A. LENSCH
LEHRSTUHL FÜR COMPUTERGRAFIK
PHILIPP LANGSTEINER (PHILIPP.LANGSTEINER@UNI-TUEBINGEN.DE)
MEHRAN HOSSEINZADEH (MEHRAN.HOSSEINZADEH@UNI-TUEBINGEN.DE)
DEBORAH KORNWOLF (DEBORAH.KORNWOLF@STUDENT.UNI-TUEBINGEN.DE)

FEBRUARY 20, 2026

MASSIVELY PARALLEL COMPUTING
GETTING STARTED GUIDE

1 Connecting to Computer Graphics Machines

The computer graphics machines are only reachable from within the WSI network. There are multiple ways to connect, the easiest way is probably via the WSI's SSH gateway `sshgw`. From there, you can use `ssh` again to then connect to any of our internal machines.

From Outside / First Login. For your first login, use the WSI's SSH gateway `sshgw`:

```
ssh <your WSI user>@sshgw.cs.uni-tuebingen.de
```

On your first login, you will be prompted to change your initial password. After signing in with your initial password, you need to type in your initial password again, and then two times a new password.

From the University / ZDV Network. If you're already in the university network (e.g., via the ZDV's WiFi or VPN), you can also connect via our SSH gateway `cgcontact`:

```
ssh <your WSI user>@cgcontact.cs.uni-tuebingen.de
```

Never work on `cgcontact` directly. This machine's entire purpose is to serve as a SSH gateway to our machines inside the WSI network. Unless you are just transferring files, only use `cgcontact` as a "proxy".

1.1 Computer Graphics Machines

Everyone is welcome to use our pool machines for their work:

`cgpool1801`..`cgpool1803`, `cgpool1900`..`cgpool1912`, `cgpoolsand1900`..`cgpoolsand1907`.

Additionally, we have several compute servers: `cluster-gpu00`..`cluster-gpu04`, `glorifolia`, `heracleum`, `myristica`, `pulsatilla`. These machines are reserved for staff, HiWis, and thesis students (groups `cgstaff`, `cgext`, `cghiwi`, and `cggpu`). Other users are not allowed to sign in on these machines.

1.2 SSH Configuration

You can simplify the SSH connection process, especially to internal machines, by creating a SSH configuration file `.ssh/config` in your `home` folder on your local machine:

```
Host cgcontact sshgw
    HostName %h.cs.uni-tuebingen.de
    User <your WSI username>
    ForwardAgent yes

Host cgpool???? cgpoolsand19?? cluster-gpu0? glorifolia heracleum myristica pulsatilla
    HostName %h.cs.uni-tuebingen.de
    User <your WSI username>
```

```
    ForwardAgent yes
    # if you are not using the VPN, you have to go through a SSH gateway
    # (skip this line if you always connect through the ZDV VPN)
    ProxyJump sshgw
```

This will allow you to connect to any machine in the list by simply writing e.g.

```
ssh cgpool1905
```

rather than

```
ssh <your WSI user>@sshgw.cs.uni-tuebingen.de
# then, from sshgw
ssh cgpool1905
```

Visual Studio Code Remote: SSH

If you want to use VS Code's SSH extension, you have to spell out the host names explicitly, rather than using wildcards like `?` or `*`. I.e. type `cgpool1800 cgpool1801` ... instead of `cgpool????`.

Do *not* run VS Code on `cgcontact`! Use an internal machine.

1.3 Login via SSH Key (CG users only)

This section only applies if your account belongs to a computer graphics group (e.g., `cgstaff`, `cgstud`). To find out, run `groups` on one of our machines or a WSI machine.

First, create a SSH key on your own machine (if you don't have one already):

```
ssh-keygen -t rsa
```

and follow the instructions on screen (generally, just confirm the defaults by pressing *Enter*).

Then, copy your *public* key into the `~/.ssh/authorized_keys` file in your `home` on our machines:

```
ssh-copy-id -i ~/.ssh/id_rsa.pub cgcontact
```

You will have to enter the password for your WSI account this time. For all following connections, your private SSH Key will be used to safely establish the connection without requiring a password.

You will *not* have access to the WSI's filesystems when logging in with an SSH Key. To access the WSI's filesystems, run `kinit` and type your password.

2 Shared Filesystems

In the WSI network, your home directory is shared across all machines and accessible to you as

```
/home/<your WSI user>/
```

No matter which machine you use, you will see the same files. This means that you can easily switch between machines without having to reconfigure anything. By default, this home is provided by the WSI.

2.1 CG Homes

If you are a member of the computer graphics group (`cgstaff`) or a student of the computer graphics group (`cgstud`), your `/home/<your WSI user>/` is on our file servers. (Run `groups` to check.)

Note that the CG homes are only available on CG machines. On other machines (such as `sshgw`), you only have access to your WSI home.

2.2 Computer Graphics File Systems

We have the following file systems:

/home (*only* for users in a `cg` group, e.g., `cgstaff` or `cgstud`) (snapshots + backup)

/graphics/opt for shared software (snapshots + backup)

/graphics/projects{,2} for long-term project data (snapshots + backup)

/graphics/scratch{,2,3,4} for public datasets and temporary data (snapshots only!)

2.3 Where to Put Your Files

Code. Keep your code in your home to make sure it is backed up. Also, consider creating a git repository for your code and committing regularly, so you can `git diff` against previous versions (even without pushing it to a remote server). You can also push it to `/graphics/projects/git_repository/` (see documentation – staff only) or a git server of your choice for an additional backup.

Temporary Data. E.g., training logs, intermediate checkpoints, virtual environments.

Please put all your temporary data into `/graphics/scratch2/students` or `/graphics/scratch3/staff`. Create a folder folder for yourself first, if necessary.

On our compute servers, there is an additional shared scratch space available at `/ceph`.

If you are not a CG student, you can use `/var/tmp` but you risk that your files may be removed at any time. (An automatic cleanup job will delete all files in the `xdg_cache_*` folders older than two months.)

Datasets. Download datasets directly into `/graphics/scratch3/datasets`, *never* into your home!

If necessary, ask your supervisor to download it for you. Make sure that your download will *not* fill up the file system – always keep enough space for others to be able to work as well!

Make sure to share datasets with others whenever possible and, if necessary, convert them into an LMDB. Do *not* run training jobs on millions of files (e.g. PyTorch `ImageFolder`).

Permanent Data. E.g., final checkpoints, results, costly processed datasets.

Once you have created some data that needs to be kept long term without any changes, move (copy, then, delete) it over to `/graphics/projects2`, so that it will be backed up.

2.4 Snapshots and Backups

Snapshots. On all our file systems, snapshots are being created at 0:00, 12:00, 16:00, and 20:00 on workdays and at 0:00 and 12:00 on the weekend. These snapshots contain the state of the entire file system at that moment in time and are kept for about three days. If you accidentally messed up or deleted a file or folder, check in `/graphics/scratch{2,3,4}/snapshots`, or, for the other file systems, ask a CG admin to get it back. :)

Backups. CG `/home`, `/graphics/opt`, `/graphics/projects` and `/graphics/projects2` are being backed up daily, early in the morning. All files visible when taking the snapshot for the backup will continue to stay around and occupy space until that snapshot is deleted (roughly two years later), so make sure to not store large temporary files on these file systems.

If you download large files which could easily be downloaded again, do not download them to your home, but directly to `/graphics/scratch{2,3,4}`, or the machine's local `/var/tmp` folder.

There are *no* backups on any of the scratch file systems.

Trash. Please empty your trash folder regularly. Trashed files in your home will still fill up our backups.

3 Rules on Computer Graphics Machines

Make sure to not disturb existing jobs. Use `cluster-smi` and `pool-smi` to find available GPUs. Once you have chosen a machine, use `htop` to check the CPU and memory usage of the machine. On many machines, other staff members or students might already be running some jobs. Please do your best to not interfere with their work. Also, try to keep machines available for running multi-GPU jobs.

Monitor your running processes. You should always check on your own processes to make sure that everything is working as intended. Use `htop` and `nvtop` to monitor CPU, GPU, and memory usage.

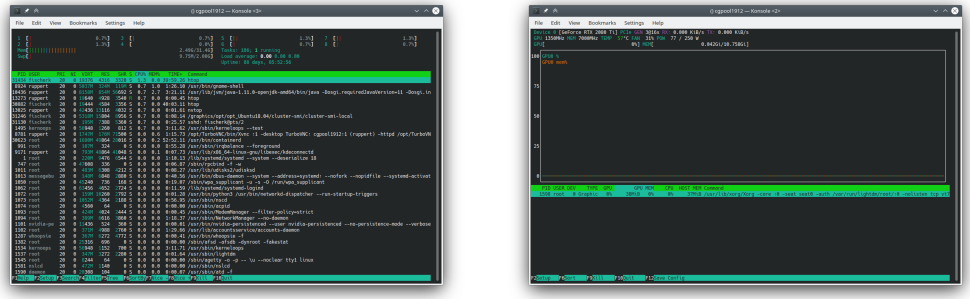

Figure 1: Use `htop` to monitor CPU and memory usage and `nvtop` to monitor GPU usage. This machine is idle and can be used freely.

4 VNC Connections

You can start a VNC server using the command `vncserver`:

```
$ vncserver

Desktop 'TurboVNC: <cgpool>:2 (<your WSI user>)' started on display <cgpool>:2

Starting applications specified in /opt/TurboVNC/bin/xstartup.turbovnc
Log file is /home/<your WSI user>/.vnc/<cgpool>:2.log
```

On the first startup, it will ask you to set a password for incoming VNC connections. It will also ask for a view-only password, which you can skip. To reset your VNC password, simply remove the `.vnc/passwd` file in your home and you will be asked to set a new password on the next start of the VNC server.

Always check for running VNC servers using `vncserver -list`, before starting a new one:

```
$ vncserver -list

TurboVNC sessions:

X DISPLAY #     PROCESS ID
:2              1125884
```

Make sure to sign out of the VNC session when you're done using it. If necessary, kill the VNC session:

```
$ vncserver -kill :2
Killing Xvnc process ID 1125884
```

Each VNC server will be accessible from port 5900 + its display id, i.e., port 5902 for display `:2` and so on. Only the ports from 5900 to 5910 are accessible from other machines (like `cgcontact`). Higher ports are blocked by the firewall.

You can now sign out of this SSH session. The VNC server keeps running in the background.

Port Forwarding. If you are connecting through a SSH gateway, you can use port forwarding to connect to the VNC server:

```
# on your own machine
ssh -L<local port>:<cgpool>:<5900+display id> sshgw
# e.g. ssh -L5902:cgpool1905:5902 sshgw
```

- Make sure to run this command in a terminal *on your own machine*.
- The command above assumes that you wrote the `~/.ssh/config` file as described in Section 1.2. Otherwise, replace `sshgw` with `<your WSI user>sshgw.cs.uni-tuebingen.de`.
- This will create a new SSH connection to `sshgw` and also forward all requests made to the `<local port>` on your own machine to the port on the pool machine. For the `<local port>`, you can choose any number between 1025 and 65535. For simplicity, you can use the same number.
- When you close that SSH connection to `sshgw`, the tunnel to the VNC server's port will also close.

If you are connected to the university network, you can also use `cgcontact` instead of `sshgw`.

Connecting with your VNC client. While the SSH tunnel is active, you can – on your machine – connect to the VNC server via `localhost:5902` (or the port number you chose). We recommend using the TurboVNC viewer, which allows to adjust the screen size, but any VNC client should work.

Figure 2: Connect to the VNC server forwarded to your local machine and type your password.

Your VNC client will ask you for the password you set during the first start of the VNC server.
You should now have access to the graphical desktop environment.

VNC Performance. Make sure to enable "Interframe comparison" in the options to reduce the amount of data that needs to be transferred to your VNC client per frame to get better performance. Also avoid using the lossless / non-JPEG modes.

GNOME Screen Lock. Never have more than one GNOME session running at a time or you will not get past the unlock screen. Always log out when you're done using a GNOME session.
Alternatively, it may work to disable the automatic screen locking.

KDE. You can start a KDE session in the VNC server with the following command:

```
vncserver -wm plasma
```

5 Transferring Files

You can transfer files via SSH using SFTP, using any of our machines: E.g., sftp://cgcontact

On Linux, simply click the link or type it into the address bar of your file manager.

If necessary, navigate to /home/<your WSI user>/, e.g. sftp://cgcontact/home/<your WSI user>.

On the command line, you can use scp, e.g.

```
scp some_file <user>@cgcontact:/home/<user>/
```

On Windows, you can use WinSCP.

Please note that you can only connect to cgcontact from the university network.

6 Customizing your Environment

Our machines run Ubuntu 24.04 with lots of software already installed. If you need something that is not yet installed, ask for it to be installed. Software that is available on the Ubuntu repositories will usually be installed, no questions asked. Software that needs to be manually (compiled and) installed, can be installed to /graphics/opt/opt_Ubuntu24.04/.

Do *not* pollute your home directory with software that could be shared with the entire group. In general, there is *no need* for local package managers like conda!

In particular, our machines come with CUDA and PyTorch pre-installed. No setup required.

If you have any questions or suggestions regarding our setup, ask cg-admin@cs.uni-tuebingen.de.

Login Shell / .bash_profile. Whenever you login to a machine, bash is running in the terminal, awaiting your commands. It can be customized by creating two hidden files in your home: .bashrc and .bash_profile. Your .bash_profile file is sourced (executed as if typed in the current shell) by bash whenever you login. It should only contain the following content (you need to create it!):

```
if [ -f ~/.bashrc ]; then
    . ~/.bashrc # does the same as "source ~/.bashrc"
fi
```

This will check for your .bashrc file and source it (execute it as if typed in the current shell). By having this file, your login shell (the shell that runs when you login via SSH) will behave the same as when you run bash manually from the command line or when you open a terminal from a graphical session. In these cases, only the .bashrc file is sourced and the .bash_profile is ignored.

Your Environment / .bashrc. You can override our default settings in your .bashrc file. E.g., to use a different version of the **CUDA Toolkit**, you can simply copy the following into your .bashrc file and adjust the version number:

```
# CUDA
export CUDA_TOOLKIT_VER=11.8.0
export CUDA_PATH="${GRAPHICS_OPT}/cuda/toolkit_${CUDA_TOOLKIT_VER}/cuda"
export PATH="${CUDA_PATH}/bin:${CUDA_PATH}/nvvm/bin:${PATH}"
export LD_LIBRARY_PATH="${CUDA_PATH}/lib64:${LD_LIBRARY_PATH}"
```

Here, ${GRAPHICS_OPT} refers to /graphics/opt/opt_Ubuntu24.04/.

You can also add further configurations depending on your needs.

The PATH variable is a list of directories in which to check for executable programs. Similarly, the LD_LIBRARY_PATH variable is a list of directories in which to check for libraries (.so). Individual paths are separated by a colon :. Make sure to always append from the front, since the list of paths is processed from left to right, and prefer using quotes (you need them if there is a space somewhere in the path).

In addition to exporting environment variables, you can also set aliases to act like programs or to set default arguments. E.g., the following alias is set by default on most modern Linux systems:

```
alias ls='ls --color=auto'
```

You might also want to enable auto-completion on the command line:

```
source /usr/share/bash-completion/bash_completion
```

Whenever you press [⇥] on the command line, `bash` will try to auto-complete the command you're writing. By default, it can complete partially written names of files and directories. If you press [⇥] twice, it will also print all the available options for you. By sourcing this file, the arguments of many programs like `git` can also be completed by pressing [⇥].

Editing text files on the command line. If you're connecting via SSH, you can use the command line editors `nano` or `vi` (VIM) to create and edit text files on the command line.

If you're not familiar with either one, `nano` is probably easier to use. To save files in `nano`, press [Ctrl]+[O] and confirm with [Enter]. To exit `nano`, press [Ctrl]+[X].

In `vi` (VIM), press [i] or [Insert] to enter edit mode and make your changes. Press [Esc] to switch back to command mode. In command mode, write `:w` and press [Enter] to save, `:q` to quit and `:q!` to force-quit without saving. You can find more documentation online.

More command line tools. More command line tools you might want to look into are:

`htop` (system monitor), `nvtop` (Nvidia GPU usage), `less` (pager - view files more easily, quit with `q`), `cat` (concatenate - easiest way to print a text file), `tmux` (terminal multiplexer - for having multiple terminals in one session and keeping your session alive when you sign out), `find` (find files), `grep` (filter by regular expressions), `sed` (stream editor), `man` (manual), `ls` (list), `cd` (change directory), `pwd` (print working directory), `touch` (create file or update its last modified date), `mkdir` (make directory), `rm` (remove), `rmdir` (remove empty directory), `df` (disk free), `ncdu` (ncurses disk usage), `git` (version control system).

Some also prefer using `zsh` over `bash`.

6.1 Virtual (Python) Environments

When using virtual environments, please consider using the packages from the system-wide environment and not re-installing everything (particularly PyTorch, the CUDA Toolkit, and CuDNN).

I.e., create an environment with

```
virtualenv --system-site-packages /path/to/your/new/environment
```

and activate it with

```
. /path/to/your/new/environment/bin/activate
```

Then,

```
python3 -m pip install <whatever else you may need here>
```

6.2 Selecting GPUs

In machines with multiple GPUs, you can select which GPU you want to use by setting the environment variable `CUDA_VISIBLE_DEVICES`. E.g.,

```
export CUDA_VISIBLE_DEVICES="2,3"
```

selects GPUs 2 and 3 to be available to any CUDA programs you run from this terminal. They will appear as if they were the only GPUs in the system and your program will see them as GPUs 0 and 1.

You might want to add the following to your `.bashrc` file, to make this more convenient:

```
cuda() {
    if [ $# -ge 1 ]
    then
        export CUDA_VISIBLE_DEVICES="$1"
    fi
    echo "CUDA_VISIBLE_DEVICES=${CUDA_VISIBLE_DEVICES}"
}
export cuda
```

Then, you can just write, e.g., `cuda 1` instead of `export CUDA_VISIBLE_DEVICES=1`.

A Using the Command Line

For your convenience, a cheat sheet of some command line utilities:

A.1 tmux

`tmux` is a terminal multiplexer which allows you to switch between multiple terminals in a single session and to keep them alive when you disconnect.

- Start a new session: `tmux`
- Attach to an existing session: `tmux a`
- Detach from a session: Ctrl+B, D
- Create a new window: Ctrl+B, C
- Create a new pane (split horizontal): Ctrl+B, "
- Create a new pane (split vertical): Ctrl+B, %
- Switch between windows (by number): Ctrl+B, 0 / 1 / 2 / ...
- Change window number: Ctrl+B, .
- Switch between sessions, windows, and panes (menu): Ctrl+B, W
- Switch between panes: Ctrl+B, ↑ / ↓ / ← / →
- Change size of current pane: Ctrl+B, Ctrl+↑ / Ctrl+↓ / Ctrl+← / Ctrl+→
- Kill current pane: Ctrl+B, X
- Move current pane to new window: Ctrl+B, !

A.2 less

`less` is a pager which allows you to scroll through text that would otherwise be too large for your terminal.

- Quit: Q
- Search (regex): /
- Search backwards (regex): ?
- Next: N
- Previous: ⇧+N
- Filter (regex): & (for large files, piping through `grep` is a lot faster)
- Save: S (only for output piped into `less`)
- Follow: ⇧+F
- Help: H

A.3 git

A few `git` commands that you may need sooner or later.

- `git clone <repo>` (clone a repository)
- `git clone --recurse-submodules <repo>` (clone a repository including submodules)
- `git submodule update --init --recursive` (initialize and update all submodules)
- `git submodule sync` (switch submodule remotes to the URL configured in the main repository)
- `git pull` (pull updates from the remote git repository)
- `git status` (check for changed files)
- `git diff` (list changes that have not yet been added)
- `git diff HEAD` (list changes since last commit)
- `git difftool HEAD` (visually show changes since last commit)
- `git add -- <files / directories>`
- `git add -- .` (add everything in current directory)
- `git add -u` (add all updated files)
- `git commit` (commit added changes)
- `git push` (push local commits to remote)
- `git log` (show commit history)
- `git checkout <branch>` (switch to a different branch)
- `git checkout -b <new branch>` (create a new branch)
- `git checkout -- <files / directories>` (revert files/directories to the last committed state)
- `git branch -a` (list all branches)
- `git stash` (stash current changes, i.e. revert the files but keep the changes around)
- `git stash apply` (apply stashed changes, i.e. apply the changes undone by stashing)
- `git stash drop` (drop stash, i.e. drop the changes kept in the stash – apply does not drop them)
- `git remote show` (show configured remote git repositories)
- `git status --ignored` (check for changed *and* ignored files – configured in `.gitignore`)
- `git clean -xdf .` (removes *all* files not tracked by `git` recursively)
- `git worktree add <path> <commit>` (checkout another branch of this repo into a second folder)
- `git worktree prune` (cleanup removed worktree folders)

To avoid committing temporary files or compiled binaries, use a `.gitignore` file.
`git status` will often propose useful commands, given the current state of the repository.